

## **MVC PHP TD5 :Architecture MVC Avancée (1/2)**

### **Sommaire**

|                                           |   |
|-------------------------------------------|---|
| <a href="#"><u>Introduction</u></a> ..... | 1 |
| <a href="#"><u>Exercice:1</u></a> .....   | 2 |
| <a href="#"><u>Exercice :2</u></a> .....  | 2 |
| <a href="#"><u>Exercice:3</u></a> .....   | 4 |
| <a href="#"><u>Exercice :4</u></a> .....  | 5 |
| <a href="#"><u>Exercice:5</u></a> .....   | 6 |
| <a href="#"><u>Exercice : 6</u></a> ..... | 6 |
| <a href="#"><u>Exercice:7</u></a> .....   | 7 |
| <a href="#"><u>Exercice : 8</u></a> ..... | 8 |
| <a href="#"><u>Exercice : 9</u></a> ..... | 8 |
| <a href="#"><u>Conclusion</u></a> .....   | 8 |

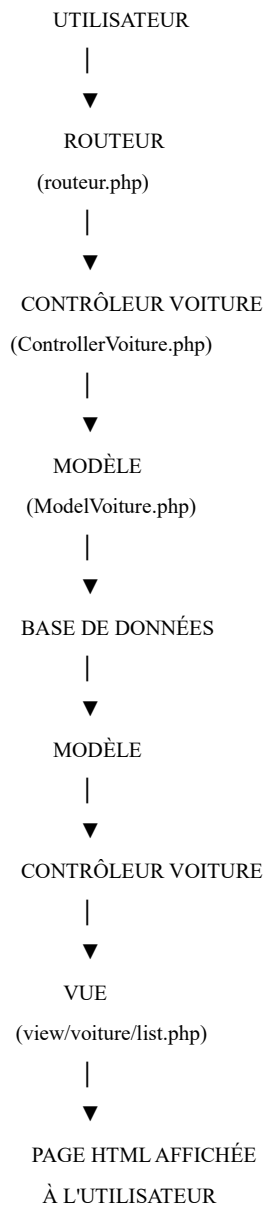
### **Introduction**

Dans ce TP , je vais poursuivre la mise en place de l'architecture MVC sur mon site de covoiturage. Je vais notamment travailler sur l'organisation du projet, la gestion des chemins de fichiers, la sécurité des données affichées et la modularisation des vues. L'objectif est de rendre mon application plus claire, sécurisée et facile à maintenir.

## **Exercice:1**

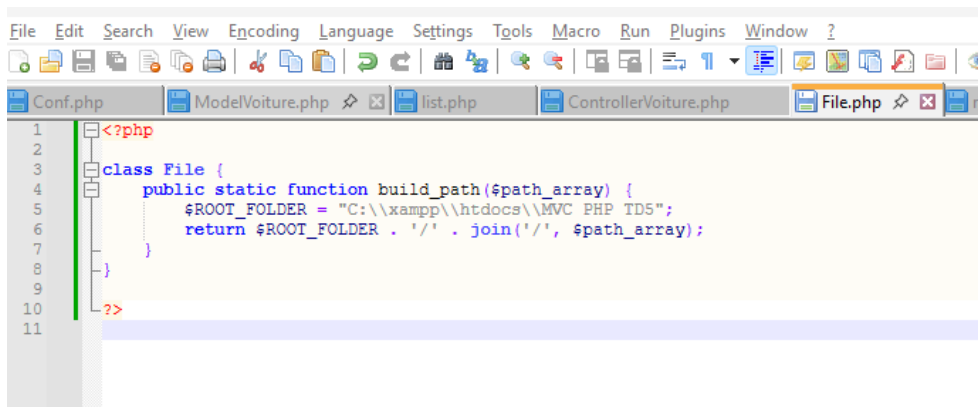
L'utilisateur envoie une requête au routeur. Le routeur identifie l'action demandée et appelle le contrôleur correspondant. Le contrôleur utilise ensuite le modèle pour récupérer ou modifier les données dans la base de données. Une fois les données récupérées, le contrôleur les transmet à la vue

### **Schéma du fonctionnement de l'architecture MVC**



## Exercice :2

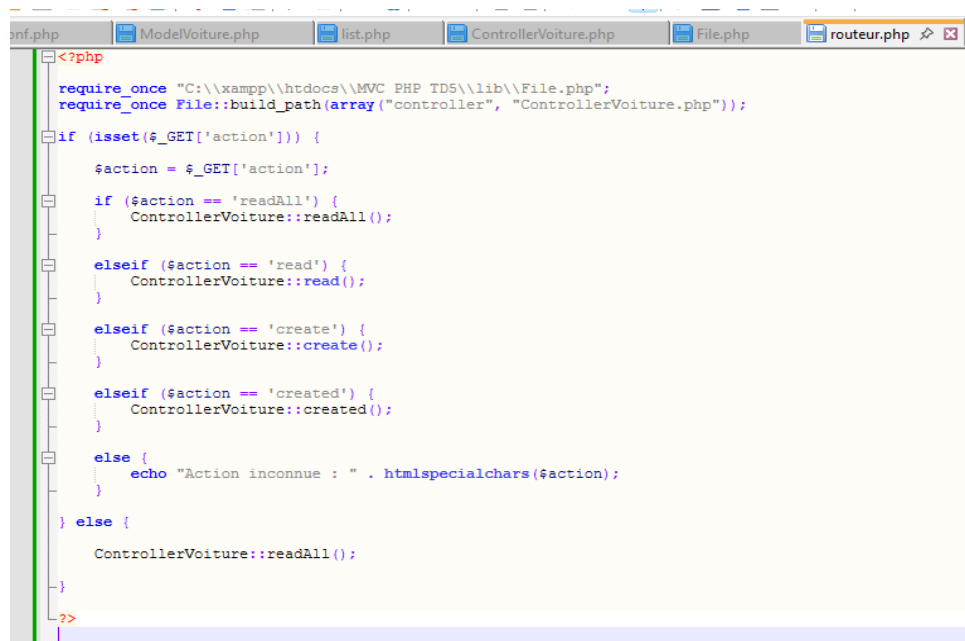
J'ai créé la classe File dans le fichier lib/File.php afin de faciliter la gestion des chemins vers mes fichiers. J'ai ensuite utilisé la fonction build\_path() dans mon routeur pour charger ControllerVoiture.php. J'ai testé mon site avec l'action readAll et tout fonctionnait correctement



```

1  <?php
2
3  class File {
4      public static function build_path($path_array) {
5          $ROOT_FOLDER = "C:\\xampp\\htdocs\\MVC PHP TD5";
6          return $ROOT_FOLDER . '/' . join('/', $path_array);
7      }
8  }
9
10 >?
11

```



```

<?php
require_once "C:\\xampp\\htdocs\\MVC PHP TD5\\lib\\File.php";
require_once File::build_path(array("controller", "ControllerVoiture.php"));

if (isset($_GET['action'])) {
    $action = $_GET['action'];

    if ($action == 'readAll') {
        ControllerVoiture::readAll();
    }

    elseif ($action == 'read') {
        ControllerVoiture::read();
    }

    elseif ($action == 'create') {
        ControllerVoiture::create();
    }

    elseif ($action == 'created') {
        ControllerVoiture::created();
    }

    else {
        echo "Action inconnue : " . htmlspecialchars($action);
    }
} else {
    ControllerVoiture::readAll();
}
>?

```

### Exercice:3

J'ai rendu mon projet portable en utilisant `__DIR__` et `DIRECTORY_SEPARATOR` afin de ne plus avoir à écrire directement le chemin de mon ordinateur. J'ai ensuite testé mon site pour vérifier qu'il fonctionnait toujours correctement.

```
<?php
class File {
    public static function build_path($path_array) {
        $DS = DIRECTORY_SEPARATOR;
        $ROOT_FOLDER = __DIR__ . $DS . "..";
        return $ROOT_FOLDER . $DS . join($DS, $path_array);
    }
}
```

```
<?php
$DS = DIRECTORY_SEPARATOR;
$ROOT_FOLDER = __DIR__;

require_once $ROOT_FOLDER . $DS . "lib" . $DS . "File.php";
require_once $ROOT_FOLDER . $DS . "controller" . $DS . "routeur.php";
```

### Exercice :4

J'ai testé une immatriculation contenant du code HTML et j'ai constaté que la balise était interprétée par le navigateur. Cela montre qu'il faut sécuriser les données affichées dans les vues.

## Liste des voitures

Immatriculation : **<h1>Hack** | Marque : clio 92 | Couleur : fiscale

### Résultat :

#### Détail de la voiture

Immatriculation :

**Hack**

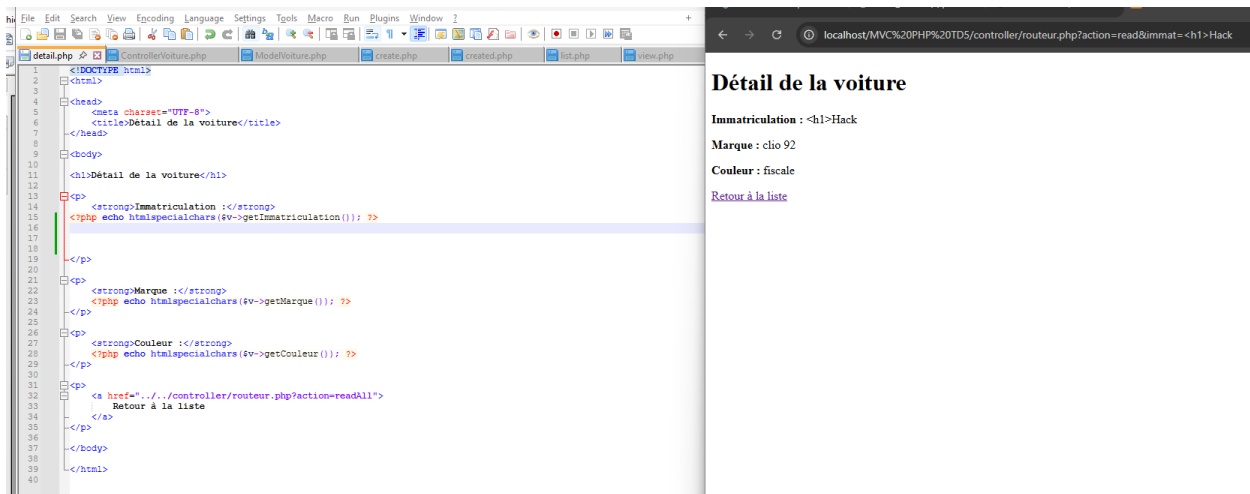
Marque : clio 92

Couleur : fiscale

[Retour à la liste](#)

## Exercice:5

J'ai sécurisé les données affichées avec `htmlspecialchars()` et les données présentes dans les URL avec `rawurlencode()`. J'ai ensuite testé mon site pour vérifier que les informations étaient correctement protégées.



## Exercice:7

J'ai créé une vue générale `view.php` qui permet de gérer la structure commune de mes pages. J'ai ensuite modifié mes vues et mon contrôleur pour utiliser cette vue générale. Cela permet d'éviter de répéter le code HTML dans chaque page et facilite la maintenance du site.



```

<?php
require_once __DIR__ . '/../model/ModelVoiture.php';

class ContrôleurVoiture {
    // Afficher toutes les voitures
    public static function readAll() {
        $tab_v = ModelVoiture::getAllVoitures();

        $controller = "voiture";
        $view = "list";
        $pagetitle = "Liste des voitures";

        require __DIR__ . '/../view/view.php';
    }

    // Afficher le détail d'une voiture
    public static function read() {
        $immat = $_GET['immat'];
        $v = ModelVoiture::getVoitureByImmat($immat);

        if ($v == false) {
            $controller = "voiture";
            $view = "error";
            $pagetitle = "Erreur";

            require __DIR__ . '/../view/view.php';
        } else {
            $controller = "voiture";
            $view = "detail";
            $pagetitle = "Détail de la voiture";

            require __DIR__ . '/../view/view.php';
        }
    }

    // Afficher le formulaire de création
    public static function create() {
        $controller = "voiture";
        $view = "create";
        $pagetitle = "Ajouter une voiture";

        require __DIR__ . '/../view/view.php';
    }

    // Créer une voiture
    public static function created() {
        $immatriculation = $_GET['immatriculation'];
        $marque = $_GET['marque'];
        $couleur = $_GET['couleur'];

        $v = new ModelVoiture(
            $marque,
            $couleur,
            $immatriculation
        );
    }
}

```

## Liste des voitures

Immatriculation : [<h1>Hack](#) | Marque : clio 92 | Couleur : fiscale

Immatriculation : [<h1>haqq](#) | Marque : clio | Couleur : rouge

Immatriculation : [AA111BB](#) | Marque : renault | Couleur : rouge

Immatriculation : [AB123CD](#) | Marque : Mercedes | Couleur : Noir

Immatriculation : [EF456GH](#) | Marque : BMW | Couleur : Blanc

Immatriculation : [gb 744t](#) | Marque : clio 3 | Couleur : noir

Immatriculation : [IJ789KL](#) | Marque : Renault | Couleur : Rouge

Immatriculation : [MNI23OP](#) | Marque : Ferrari | Couleur : Jaune

Immatriculation : [QR456ST](#) | Marque : Audi | Couleur : Gris

Immatriculation : [UV789WX](#) | Marque : Peugeot | Couleur : Bleu

Immatriculation : [ZZ999ZZ](#) | Marque : Toyota | Couleur : Vert

[Ajouter une voiture](#)

## Exercice : 8

J'ai ajouté un menu avec des liens vers les voitures, les utilisateurs et les trajets. J'ai également ajouté un footer commun à toutes les pages grâce à la vue générale `view.php`.

```

<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title><?php echo $pagetitle; >></title>
</head>
<body>
<nav>
<a href="/MVC%20PHP%20TD5/index.php?action=readAll">
    Voitures
</a>
<a href="/MVC%20PHP%20TD5/index.php?action=readAll&controller=utilisateur">
    Utilisateurs
</a>
<a href="/MVC%20PHP%20TD5/index.php?action=readAll&controller=trajet">
    Trajets
</a>
</nav>
<hr>
<?php
$filespath = File::build_path(array("view", $controller, "$view.php"));
require $filespath;
?>
<hr>
<p style="border: 1px solid black; text-align:right; padding-right:1em;">
    Site de covoiturage
</p>
</body>
</html>

```

[Voitures](#) [Utilisateurs](#) [Trajets](#)

## Liste des voitures

Immatriculation : [<h1>Hack](#) | Marque : clio 92 | Couleur : fiscale

Immatriculation : [<h1>haqq](#) | Marque : clio | Couleur : rouge

Immatriculation : [AA111BB](#) | Marque : renault | Couleur : rouge

Immatriculation : [AB123CD](#) | Marque : Mercedes | Couleur : Noir

Immatriculation : [EF456GH](#) | Marque : BMW | Couleur : Blanc

Immatriculation : [gb 744 t](#) | Marque : clio 3 | Couleur : noir

Immatriculation : [IJ789KL](#) | Marque : Renault | Couleur : Rouge

Immatriculation : [MN123OP](#) | Marque : Ferrari | Couleur : Jaune

Immatriculation : [QR456ST](#) | Marque : Audi | Couleur : Gris

Immatriculation : [UV789WX](#) | Marque : Peugeot | Couleur : Bleu

Immatriculation : [ZZ999ZZ](#) | Marque : Toyota | Couleur : Vert

[Ajouter une voiture](#)

Site de covoiturage

## Conclusion

Grâce à ce TP, j'ai pu approfondir l'utilisation de l'architecture MVC dans mon projet. J'ai appris à mieux organiser mes fichiers, à rendre les chemins plus portables, à sécuriser les données affichées et à éviter la répétition de code dans mes vues. Ces modifications permettent de rendre mon site de covoiturage plus propre, plus sécurisé et plus facile à faire évoluer.